HL-Spot — User guide

Version 1.0.6

HL-Spot is a trading bot for **Hyperliquid**: spot and perpetual (perp) markets, several pairs, automatic and manual orders. It runs **on your own computer** (Windows, Linux or Docker) and you control it from your web browser.

> **Website**: https://hlspot.com
> **Download**: https://github.com/HL-Spot-Bot/HL-Spot-Bot
> **Support**: HL-spot@cmails.eu

---

1. Before you start

You need:

- a computer with a 64-bit x86 processor (x86_64) running **Windows** or **Linux**, or any machine with **Docker**;
- or a **Flux** account, to run it in the cloud (section 16);
- or a **rented server (VPS)** running Linux or Windows, accessed over HTTPS (section 17);
- a **Hyperliquid** account with funds (your main wallet);
- a wallet able to **"sign a message"** with your main wallet address. A browser extension (MetaMask, Rabby...) is the simplest: the bot opens it for you. Any other wallet works by copy and paste;
- an Internet connection.

2. Security: what you must know

- HL-Spot only accepts an **API wallet** key from Hyperliquid. **Never enter the private key of your main wallet.** An API wallet can trade for you but cannot withdraw your funds.
- The API wallet key is **encrypted** on your computer. It is **never sent** to the license server.
- Your main wallet is used only to **sign messages** (account creation, forgotten password, BTC address, account deletion). A signed message is not a transaction: it costs nothing and moves no funds.
- The bot's web page is protected by your password. Prefer to use it from the bot's machine (`http://localhost:60000`): from another machine of your network, the page is not encrypted (see section 15).

3. Create your Hyperliquid API wallet

1. Go to **https://app.hyperliquid.xyz/API** and connect your main wallet.
2. Give the API wallet a name, then generate it.
3. **Copy the private key** shown and keep it in a safe place: Hyperliquid shows it only once.
4. Authorize the API wallet (signature with your main wallet).
5. Note the **expiry date** of the API wallet shown by Hyperliquid.

You will enter in HL-Spot: your **main wallet address** (0x...) and the **API wallet private key**.

4. Install HL-Spot

Download the file for your system and its `.sha256` file from https://github.com/HL-Spot-Bot/HL-Spot-Bot. The `.sha256` file lets you check that the download is intact.

Windows

1. Unzip `HL-Spot-1.0.6-prod-windows-x64.zip`.
2. In the unzipped folder, run **`HL-Spot.exe`**. A console window opens: keep it open while the bot runs.
3. Open **http://localhost:60000** in your browser.

Linux (Ubuntu 22.04, 24.04, 26.04, Debian 12 or newer)

```
unzip HL-Spot-1.0.6-prod-linux-x64.zip
cd HL-Spot-1.0.6-prod-linux-x64
./hl-spot
```

Then open **http://localhost:60000** in your browser.

Docker

```
docker load -i HL-Spot-1.0.6-prod-docker-x64.tar.gz
docker run -d --name hl-spot --restart unless-stopped -p 60000:60000 \
       -e TZ=Europe/Paris -v hl-spot-data:/data hl-spot:1.0.6
```

- `-p 60000:60000` is required to reach the web page.
- `TZ` sets the time zone (UTC if omitted).
- Your data stay in the `hl-spot-data` volume, even if the container is removed or the image updated.
- `docker stop hl-spot` stops the bot cleanly.

Then open **http://localhost:60000** (or `http://<machine address>:60000`).

Where your data are kept

Your data (settings, encrypted key, database, log) are kept **outside the program**:

System	Folder
Windows	`%LOCALAPPDATA%\HL-Spot`
Linux	`~/.local/share/hl-spot`
Docker	the `/data` volume

Reinstalling or updating the program never touches them.

5. First launch

The first page is **First launch**. Choose:

- **Create an account** if you are new;

- **I already have an account** if you already have an HL-Spot account (new computer, reinstallation).

Create an account

1. **Wallet address**: your main wallet address (0x...).
2. **API wallet private key**: the key copied in section 3. It is checked with Hyperliquid before anything else.
3. **Password**: at least 8 characters, with an uppercase letter, a lowercase letter, a digit and a special character. It protects both your HL-Spot account and the bot's web page.
4. **Main wallet signature**:
    - **Sign with my wallet**: the bot opens your wallet extension; check the address and confirm the signature;
    - or **Get the message to sign**: copy the message as is into your wallet, sign it, and paste the signature (0x...). The message is valid for 15 minutes.
5. Click **Create the account**.

Your **free trial of 7 days** starts. It is granted only once per wallet and per installation. Trading starts once your Hyperliquid account is verified.

I already have an account

Enter your wallet address and your password. This installation is registered and the previous one is released (**one installation change per 30 days**).

---

6. Using the bot

The menu at the top gives access to:

Page	Use
📊 Dashboard	one tab per spot pair: its cycles and its market status
📈 Statistics	spot and perp cycle results, by period
🧩 Traded pairs	spot and perp pairs configured for the bot, and their settings
🟣 Perp cycles	entries, take-profits, stop-losses and closes of perp pairs
🖐 Manual orders	place an order by hand; the bot then follows the cycle like the others
🌐 Hyperliquid pairs	list of Hyperliquid spot and perp pairs; add a pair to the bot from here
⚙️ Settings	global settings (applied without restart, except port and listening address)
📝 Log	errors (and warnings if enabled)
🔑 Hyperliquid account	wallet, API wallet key, expiry date
📜 License	license, subscription, payment, installation, account deletion

The bot trades only if **your Hyperliquid account is verified** and **your license is valid.** Without a valid license, only the Hyperliquid account and License pages are available.

When the bot stops trading (license ended, API wallet expired or refused), it **does not touch orders and positions already open**: they remain your responsibility.

The following subsections explain how the bot decides, then every field of the **Traded pairs** and **Settings** pages.

6.1 Market analysis: BULL, BEAR or RANGE

Before each buy (spot) or entry (perp), the bot analyses **the pair itself**, on the candles of that pair:

1. It fetches the last **Number of candles fetched** candles (setting `LIMIT`) of the pair's **Candle interval** (empty = the global **Candle interval** setting).
2. The **current price** is the close of the most recent candle.
3. It computes three moving averages of the closes: **MA4**, **MA8** and **MA12** (their number of candles is set in **Settings → Market analysis**).
4. It decides the market type, in this order:
    - **RANGE** if the MA12 is flat: over the last **MA12 periods checked**, the MA12 has moved by at most **MA12 RANGE threshold (%)** between its lowest and highest value;
    - otherwise **BULL** if MA4 > MA8 > MA12;
    - otherwise **BEAR** if MA4 < MA8 < MA12;
    - otherwise **RANGE**.
5. It also computes the **range**: the highest and lowest close over the last **RANGE - range periods** candles. Its amplitude is high − low.

Each pair then uses **its own settings for the detected market type** (BULL, BEAR or RANGE block of the pair). If the analysis fails (Hyperliquid unreachable), nothing is placed and the bot tries again at the next pass.

6.2 Spot cycle, step by step

A spot cycle is one buy followed by one sell of the same quantity.

1. **When.** The bot checks each enabled spot pair every **Short pause of the buy loop (min).** A buy is attempted when:
    - the pair is not in a pause (see step 6);
    - since the previous attempt on this pair, at least the **smallest** of the pair's three **Interval between buys (min)** values (BULL, BEAR, RANGE) has passed, whatever the current market. The first attempt after the bot starts waits for **Delay before the first buy (min).**
2. **Allowed?** Buys must be enabled globally (**Buys enabled (global)**) **and** in the pair's block of the current market (**Buys enabled**). If not, the attempt counts but nothing is placed.
3. **Prices.** With *P* = current price:
    - buy price = *P* + **Buy offset**; target sell price = *P* + **Sell offset**;
    - with the **Offset unit** `abs`, offsets are in USDC; with `pct`, in % of *P*;
    - **in a RANGE market**, the offsets are **dynamic**: buy = *P* − *d*, sell = *P* + *d*, with *d* = range amplitude × **RANGE - % of the range used** / 100 / 2. The static RANGE offsets of the pair are used only if the range cannot be computed (amplitude 0).
4. **Amount.** According to **Amount of each buy** in the block of the current market:
    - **% of balance**: amount = this % × the **available** USDC (not already held by open orders);
    - **Fixed USDC**: amount = this value, at least 10 USDC + Hyperliquid buy fees (10.01 USDC with the base fees). If the available USDC is lower, **no buy is placed** and an error is

recorded in the log;
    - quantity = amount / buy price, rounded **down** to the size step of the pair (with Fixed USDC, one more step if rounding takes the value below the minimum);
    - if the order value is below **Minimum order value (USDC)** (at least 10 USDC, Hyperliquid minimum), the buy is refused and the log shows "Value too low".
5. **Order.** A limit buy order is placed at the buy price. The cycle appears on the Dashboard as **Pending buy**, with the target sell price already recorded.
6. **Pause.** After every attempt, placed or not, the pair waits **Pause after attempt (min)** of the current market block.
7. **Buy filled.** The bot learns it from the Hyperliquid history, fetched every **Hyperliquid fetch interval (min)**. The cycle becomes **Pending sell**. A partially filled buy that is still open stays **Pending buy**.
8. **Sell.** The sell loop (every **Sell loop interval (s)**) places a limit sell order at the **target sell price recorded at step 3**. It sells the quantity actually received: Hyperliquid takes the buy fee in the bought token, so the bot sells the bought quantity minus that fee, rounded down to the size step. A small remainder may stay in your wallet; a later cycle sells it when the balance allows.
    - The sell price is not recalculated. If the market is already above it, the sell is filled immediately at the market price (better than planned).
    - If the balance is not enough, the bot tries again; after 3 attempts the problem is written as an **error** in the log.
9. **Sell filled.** The cycle becomes **Completed**. The profit is calculated with the real sell price and quantity and the real fees: quantity sold × (sell price − buy price) − buy fee − sell fee.

The **Sells enabled** switches currently have no effect: once a buy is filled, its sell is always placed.

Worked example (RANGE). Current price 85,000; over the last 20 candles the highest close is 85,200 and the lowest 84,770: amplitude 430. With **RANGE - % of the range used** = 75: *d* = 430 × 75 / 100 / 2 = 161.25. Buy at 85,000 − 161.25 = 84,838.75; target sell at 85,000 + 161.25 = 85,161.25. With **% of USDC balance** = 5 and 400 USDC available: 20 USDC, so 20 / 84,838.75 = 0.0002357 of the base token, rounded down to the pair's size step.

6.3 Perp cycle, step by step

A perp cycle is one entry (long or short), then an exit by take-profit, stop-loss or close.

1. **When.** Same rules as spot: each enabled perp pair is checked every **Short pause of the buy loop (min)**; pause after each attempt (**Pause after attempt (min)** of the current market block) and smallest of the three **Interval between entries (min)**.
2. **Before any entry, at every pass**, the bot applies the current **Direction** to the cycles already open on the pair:
    - direction **none**: entries not yet filled are cancelled; open positions follow **Direction set to "none" with an open position** (`keep_tp_sl` = leave the take-profit and stop-loss in place; `close_market` / `close_limit` = close the position);
    - direction **opposite** to an open cycle (for example `short` while a long is open): an entry not yet filled is cancelled, an open position is closed according to **Close on a reversal** (`market` = market order; `limit` = limit order at the current price).
3. **Which side.** `long` or `short`: that side. `none`: no entry. `both`: according to **Rule of the "both" direction**:
    - `first_filled`: a long entry and a short entry are placed; the first filled cancels the other;

    - `range_position`: long if the price is in the lower half of the range, short in the upper half; outside a RANGE market, no entry;
    - `alternate`: the opposite side of the pair's previous cycle.
4. **Funding filter.** No long if the funding rate is above +**Funding threshold (%)**; no short if it is below –threshold. If the funding is unavailable, no entry.
5. **Leverage and margin.** If needed, the bot sets the pair's **Leverage** and **Margin mode** on Hyperliquid before the entry.
6. **Prices and size.** With *P* = current price: entry = *P* + entry offset of the side, take- profit = *P* + take-profit offset of the side (USDC or % according to **Offset unit**). Margin according to **Margin of each entry**: **% of margin** × available margin, or **Fixed USDC** (margin × leverage of at least 10 USDC + fees; if the available margin is lower, no entry is placed and an error is recorded in the log). Size = margin × leverage / entry price, rounded down.
7. **Entry.** Limit order at the entry price. When it is filled, the bot places a **take-profit** (limit, reduce-only) at the recorded price and a **stop-loss** (stop market, reduce-only) at **Stop-loss (% of the entry price)** from the real entry price.
8. **Exit.** The cycle ends when the take-profit, the stop-loss or a close is filled. Market and stop market orders accept a deviation of at most **Slippage of market orders (%)**.

Follow the perp cycles on the 🟣 **Perp cycles** page.

6.4 Traded pairs page

The 🧩 **Traded pairs** page lists the pairs configured for the bot.

- **Add a pair**: on 🌐 **Hyperliquid pairs**, click **Add** on the pair's line. A new pair is **disabled**; its spot settings are pre-filled with the BULL / BEAR / RANGE defaults of the **Settings** page. Perp settings must be entered.
- **Edit**: opens the pair's settings: a general part, then one block per market type (BULL, BEAR, RANGE). **Save** checks every value; wrong fields are highlighted.
- **Configuration** column: **complete**, or the number of fields **to complete**. A pair can be enabled only when it is complete.
- **Enable / Disable**: only enabled and complete pairs are traded. Only USDC-quoted spot pairs can be enabled. A disabled pair places no new entry, but **its open cycles continue until they close**.
- **Delete**: possible only when the pair has no cycle in progress. Disable it first and wait for its cycles to end.
- Changes apply at the next pass of the loop concerned, without restart.

Offsets — buy or entry price = current price + offset; sell or take-profit price = current price + offset. A negative offset is below the current price.

Spot pair settings

General part:

Field	Meaning
Offset unit	`abs` = offsets in USDC; `pct` = offsets in % of the current price
Candle interval	candles of the market analysis of this pair; empty = global **Candle interval**
RANGE - % of the range used	dynamic offsets in RANGE = ± (range amplitude × this %) / 2

One block for BULL, one for BEAR, one for RANGE:

Field	Meaning
Buys enabled	buys allowed when this market type is detected
Sells enabled	currently no effect: sells are always placed
Buy offset	buy price = current price + this offset (usually negative); in RANGE, replaced by the dynamic offset
Sell offset	target sell price = current price + this offset; in RANGE, replaced by the dynamic offset
Amount of each buy	**% of balance**: share of the available USDC used for each buy; **Fixed USDC**: same amount for each buy, at least 10 USDC + Hyperliquid buy fees
Pause after attempt (min)	wait after each buy attempt in this market
Interval between buys (min)	minimum time between two buy attempts; the smallest of the three blocks is used

Perp pair settings

General part:

Field	Meaning
Offset unit	`abs` = USDC; `pct` = % of the current price
Candle interval	as for spot
Leverage	limited to the maximum leverage of the asset
Margin mode	`cross` or `isolated` (some assets require `isolated`)
Stop-loss (% of the entry price)	stop market order at this % of the real entry price
Funding threshold (%)	no long if funding > +threshold; no short if funding < –threshold
Slippage of market orders (%)	maximum deviation accepted on market and stop market orders
Rule of the "both" direction	`first_filled`, `range_position` or `alternate` (see 6.3); required as soon as a block uses `both`
Close on a reversal	`market` or `limit`
Direction set to "none" with an open position	`keep_tp_sl`, `close_market` or `close_limit`

One block for BULL, one for BEAR, one for RANGE:

Field	Meaning
Direction	`long`, `short`, `both` or `none` (no entry)
Long entry offset / Long take-profit offset	the take-profit must be above the entry
Short entry offset / Short take-profit offset	the take-profit must be below the entry
Margin of each entry	**% of margin** available or **Fixed USDC** (margin × leverage of at least 10 USDC + fees); same for long and short
Pause after attempt (min)	wait after each entry attempt in this market
Interval between entries (min)	minimum time between two entry attempts; the smallest of the three blocks is used

6.5 Settings page

⚙️ **Settings** holds the global settings. A value changed here is saved and applied immediately (port and listening address: at the next restart). **Default value** goes back to the original value. The wallet address and the API wallet key are not set here (page 🔑 **Hyperliquid account**).

Operating mode

Setting	Default	Meaning
Simulation mode (DRY_RUN)	no	the bot reads Hyperliquid normally but sends no order and records no cycle

Market analysis — common to all pairs (see 6.1)

Setting	Default	Meaning
Candle interval	1h	candles used when a pair has no candle interval of its own
MA4 period / MA8 period / MA12 period	4 / 8 / 12	number of candles of each moving average
MA12 RANGE threshold (%)	0.25	maximum MA12 variation to detect a RANGE market
MA12 periods checked	5	number of periods over which the MA12 is checked
Number of candles fetched	100	must cover the largest period used (MA12 + periods checked, range periods)

Order activation

Setting	Default	Meaning
Buys enabled (global)	yes	master switch: off = no buy on any spot pair
Buys in BULL / BEAR / RANGE	yes / no / yes	default value for new spot pairs
Sells enabled (global), Sells in BULL / BEAR / RANGE	—	currently no effect

BULL / BEAR / RANGE market — defaults for new spot pairs: buy and sell offsets (USDC), % of USDC balance, buy amount (% of USDC balance or fixed amount; default: %), fixed amount (default 15 USDC), pause after an attempt, interval between buys. They pre-fill a spot pair when it is added; **changing them does not change pairs already added**. The RANGE block also holds:

Setting	Default	Meaning
RANGE - range periods	20	number of candles used for the high and low of the range (all pairs)
RANGE - % of the range used	75	default value for new spot pairs

Defaults: BULL buy 0 / sell +1000, 3 %, pause 10 min, interval 360 min; BEAR buy −1000 / sell 0, 3 %, pause 10 min, interval 360 min; RANGE buy −400 / sell +400, 5 %, pause 10 min, interval 180 min.

Orders

Setting	Default	Meaning
Minimum order value (USDC)	10	smaller orders are not placed (Hyperliquid minimum: 10)

Hyperliquid fees (read-only): your fees are read from Hyperliquid when the bot starts, then every hour, separately for **spot** and **perp** (maker = limit order, taker = market order). They take into account your volume tier, your staked HYPE and your referral discount. They are only used to estimate the fees of a trade whose real fees are not known yet: profits always use the real fees of each execution. The **Refresh** button reads them again immediately. Until they have been read, the bot uses the Hyperliquid base rates (spot: maker 0.040%, taker 0.070%; perp: maker 0.015%, taker 0.045%).

Timing and synchronization

Setting	Default	Meaning
Hyperliquid fetch interval (min)	10	how often the open orders, fills and history are fetched: a fill is seen at most this long after it happens
Delay before the first buy (min)	0	after the bot starts; applied at the next start
Short pause of the buy loop (min)	1	wait between two checks of the buy interval, and after an error
Sell loop interval (s)	120	wait between two passes of the sell loop

Telegram notifications — see section 9.

Web interface

Setting	Default	Meaning
Language	English	language of the web interface and Telegram messages
Theme	Dark	dark or light display
Listening address	0.0.0.0	0.0.0.0 = reachable from the local network; 127.0.0.1 = this computer only (restart)
Web interface port	60000	applied at restart
Pair list cache (s)	43200	the Hyperliquid pair list is kept 12 h and refreshed in the background
Delay between catalog requests (ms)	150	pause between two requests when loading the pair list
Session duration (h)	12	applies to the next logins
Failed logins before lockout	5	per IP address
Lockout duration (min)	15	

Log file

Setting	Default	Meaning
Record warnings	no	errors are always recorded; warnings only if enabled (section 12)

7. License and subscription

Prices

Subscription	Price
7 days	2 $
30 days	6 $

The period paid is added to the end of your current license (or starts on the payment date if the license has ended).

How to pay

On the **License** page, **Subscription and payment** block:

1. choose the subscription, the token and the network;
2. the bot shows the **exact amount**, the **receiving address** and the address you must **pay from.** The amount is valid for **1 hour** (less if the price moves by more than 10 %);
3. send **exactly this amount**, on **this network**, **from the wallet of your HL-Spot account**;
4. the payment is recognized automatically (a few minutes depending on the network) and the license is extended.

The tokens and networks offered are those shown on the License page.

Rules — read them before paying:

- pay **exactly** the amount requested, neither more nor less; **network fees are yours**;

- pay **from your account's wallet** (for BTC: from your declared BTC address);
- pay on the network shown, while the amount is valid;
- a payment that does not follow these rules (unknown address, different amount, other network, after the validity) **is lost: no refund.**

Paying in BTC

Before your first BTC payment, declare your **BTC address** on the License page (proof by a signature of your main wallet). The payment is recognized only if it is sent **from this BTC address**: in your BTC wallet, choose this address as the source of the payment ("coin control").

License checks

- The license is checked with the server **every 6 hours.**
- If the server cannot be reached, the bot continues until the known end date of the license.
- **24 hours before the end**: warning on the web pages and by Telegram.
- After the end: **24 hours of grace** (trading continues, with a warning), then trading stops.
- Do not set your computer's clock back: a clock moved back by more than 5 minutes stops trading until the next successful check.

---

8. API wallet: expiry and replacement

- The **Hyperliquid account** page shows the expiry date of your API wallet. You can enter it yourself if needed.
- During the **last 7 days**: banner on the web pages and a daily Telegram message.
- At the expiry date, trading stops. Create a new API wallet (section 3) and enter its key on the **Hyperliquid account** page: trading restarts without restarting the program.
- The new key must belong to the **same main wallet**: the wallet address cannot be changed.
- The bot checks with Hyperliquid at startup and every 24 hours that the API wallet still belongs to your wallet.

---

9. Telegram notifications

In **Settings → Telegram notifications**:

1. create a Telegram bot with **@BotFather** and copy its token;
2. get your chat ID (for example with **@userinfobot**);
3. enter the token and the chat ID, then enable notifications and choose the messages (orders placed, buys filled, completed cycles, errors, daily summary).

---

10. Using HL-Spot on another computer

Install the bot on the new computer and choose **I already have an account** at first launch. The old installation is released. **One change per 30 days.** The License page shows the date of the next possible change.

11. Forgotten password

On the login page, click **Forgot your password?**. Prove that you own the wallet by a **signature of your main wallet**, then choose a new password.

12. Log

The 📝 **Log** page shows the errors recorded by the bot (and the warnings if **Settings → Log file → Record warnings** is enabled). The file is limited to 1 MB: the oldest entries are removed. You can filter, download the file (useful for support) and clear it.

13. Delete your account

License page, 🗑️ **Delete my account**: password + signature of your main wallet.

- Your HL-Spot account is deleted **permanently**.
- The remaining license time is **lost and not refunded**.
- The free trial is **not** granted again for this wallet.
- On this computer, the wallet address, the API wallet key and the password are erased; the trading history is kept.

14. Updates and backup

When a new version is published, the bot shows a banner at the top of every page (check at start-up, then every 6 hours):

- **blue banner**: a new version is available. The **Ignore this version** button hides it until the next version;
- **red banner**: **important** update (a security fix, for example), to install as soon as possible; it cannot be ignored.

If Telegram notifications are enabled (section 9), a message is sent only once per new version. The bot never downloads or installs anything by itself: the banner link opens the page of the version on GitHub.

- **Update**: install the new version (unzip it, or load the new Docker image); your data are kept (section 4). On Flux: section 16; on a VPS: section 17.
- **Backup**: copy the data folder (section 4). The `.env` file and the `secret.key` file go **together**: the encrypted values of `.env` and of the database cannot be read without `secret.key`. If `secret.key` is lost, these values (API wallet key, Telegram token…) must be entered again.

15. Access from another machine

By default the web page listens on all network interfaces, port **60000** (**Settings → Web interface**, applied at restart). From another machine of your network: `http://<bot machine address>:60000`.

On the network, the page is **not encrypted**: enter your API wallet key and your password preferably from the bot's machine. Never expose port 60000 directly to the Internet.

To reach the bot from the Internet (VPS), follow section 17: HTTPS access is required.

16. Running HL-Spot on Flux

Flux is a decentralized cloud: it rents containers on servers (nodes) run by third parties. HL-Spot can run there day and night without your computer. Flux is independent of HL-Spot and is paid to Flux.

Your API wallet key on Flux: read this first

On Flux, the bot's data (`/data`: the encrypted API wallet key **and** the `secret.key` file that decrypts it) are on servers run by other people, copied on 3 nodes. You choose the type of app when you deploy it:

- **Enterprise app** (ArcaneOS nodes) — recommended: Flux states that node operators cannot access the app's data (encrypted disk, restricted root access) and that environment variables stay private.
- **Ordinary app**: a node operator can read `/data`, so your API wallet key; environment variables, including the first-launch code, can be read by anyone. An API wallet cannot withdraw your funds, but whoever holds its key can trade on your account. At your own risk.

Whatever the type, you can revoke the API wallet on Hyperliquid at any time (section 3).

App settings

Flux field	Value
Image	`olivier1246/hl-spot:1.0.6`
Port and container port	`60000`
Container data	`g:/data` (**required**)
CPU	0.2
RAM	300 MB (increase it if the app restarts for lack of memory)
SSD	3 GB
Instances	3 (Flux minimum)
Environment	`HL_SPOT_SETUP_CODE=<your code>` (**required**), `TZ=Europe/Paris` (optional)

- `g:/data`: only **one** instance runs the bot; the 2 others keep a synchronized copy of the data and take over if it stops. Never use another value: the bot would run on 3 machines at once and place each order 3 times.
- `HL_SPOT_SETUP_CODE`: a code of 12 characters or more, different from your password. Without it, anyone who finds the address of the app could create the account before you.

First launch on Flux

1. Open the **https** address that Flux gives for the app.
2. Follow section 5, and enter your code in the **First-launch code** box.
3. Use a browser with your wallet extension: the bot signs the message with it.

Good to know

- The installation follows the data: when the bot moves to another node, this does not count as an installation change (section 10).

- After a move, the bot restarts from the synchronized copy: check your open orders on Hyperliquid.
- **Update**: change the image to the new version in the app settings on Flux.
- The page is reachable from the Internet: it is protected by your password. Choose a strong one.

---

17. Installing HL-Spot on a VPS (HTTPS access)

On a rented server (VPS), the bot's page is reached from the Internet. It must go through **HTTPS**: without encryption, your password and your API wallet key travel in clear text, and Internet robots constantly probe open ports. The bot shows a warning banner and records an error in the log when it receives an HTTP request coming from the Internet.

How it works: an HTTPS proxy, **Caddy**, installed on the same machine, receives the connections on port 443 and passes them to the bot. Caddy obtains and renews a **Let's Encrypt** certificate by itself:

- on the **public IP address** of the VPS (default): short-lived certificate (about 6 days), renewed automatically;
- or on your **domain name** (e.g. `bot.example.com`), whose DNS record must point to the IP address of the VPS.

Ports to open: **80** (certificate) and **443** (HTTPS), in addition to your administration access (SSH, Remote Desktop). **The bot's port 60000 must never be open.** If your hosting provider has its own firewall, open 80 and 443 there too.

The files you need are in the `vps/` folder of the repository https://github.com/HL-Spot-Bot/HL-Spot-Bot.

Linux (Ubuntu 22.04 or newer, Debian 12 or newer)

```
curl -fLO https://raw.githubusercontent.com/HL-Spot-Bot/HL-Spot-Bot/main/vps/install-linux.sh
sudo sh install-linux.sh <public IP address or domain name>
```

The script installs Caddy, downloads HL-Spot and checks its SHA-256 checksum, creates the `hl-spot` service (automatic start, data in `/var/lib/hl-spot`), and configures HTTPS and the `ufw` firewall (SSH, 80, 443). At the end it shows the **first-launch code**, kept in `/etc/hl-spot/hl-spot.env`.

The bot runs as the systemd service `hl-spot`: it starts with the machine and, if it stops, it is restarted automatically after 10 seconds. Service commands:

```
sudo systemctl status hl-spot      # status
journalctl -u hl-spot -f           # live log
sudo systemctl restart hl-spot     # restart
sudo systemctl stop hl-spot        # stop (until the next start of the machine)
sudo systemctl start hl-spot       # start
```

- **Update**: download the new script and run it again with the version as second parameter (e.g. `sudo sh install-linux.sh <address> 1.0.7`). Your data is kept.

Docker

In a folder, download the files:

```
for f in docker-compose.yml Caddyfile.ip Caddyfile.domain; do
  curl -fLO https://raw.githubusercontent.com/HL-Spot-Bot/HL-Spot-Bot/main/vps/$f
done
```

Create a `.env` file in this folder:

```
HL_SPOT_SITE=<public IP address or domain name>
HL_SPOT_CADDYFILE=Caddyfile.ip
HL_SPOT_SETUP_CODE=<your code, 12 characters or more>
TZ=Europe/Paris
```

(`HL_SPOT_CADDYFILE=Caddyfile.domain` for a domain name), then run `docker compose up -d`.

- Only Caddy publishes ports (80 and 443). Never publish the bot's port 60000: Docker bypasses the `ufw` firewall for the ports it publishes.
- With Docker, the bot's listening address stays **0.0.0.0** (Caddy reaches it through Docker's internal network).

Windows Server

PowerShell commands, run as administrator.

1. Install HL-Spot (section 4) in a fixed folder, for example `C:\HL-Spot` (`HL-Spot.exe` directly in this folder).

2. First-launch code (12 characters or more):

```
[Environment]::SetEnvironmentVariable('HL_SPOT_SETUP_CODE', '<your code>', 'Machine')
```

3. Automatic start of the bot, under your Windows account (even with no open session):

```
$a = New-ScheduledTaskAction -Execute 'C:\HL-Spot\HL-Spot.exe' -WorkingDirectory 'C:\HL-Spot'
$t = New-ScheduledTaskTrigger -AtStartup
Register-ScheduledTask -TaskName 'HL-Spot' -Action $a -Trigger $t -User $env:USERNAME -Password '<Windows password>'
Start-ScheduledTask -TaskName 'HL-Spot'
```

4. Caddy: download `caddy_<version>_windows_amd64.zip` from https://github.com/caddyserver/caddy/releases and put `caddy.exe` in `C:\Caddy`. Copy `Caddyfile.ip` (or `Caddyfile.domain`) from the `vps/` folder to `C:\Caddy\Caddyfile`, then:

```
(Get-Content C:\Caddy\Caddyfile) -replace '\{\$HL_SPOT_SITE\}', '<IP address or domain>' | Set-Content C:\Caddy\Caddyfile
sc.exe create caddy start= auto binPath= "C:\Caddy\caddy.exe run --config C:\Caddy\Caddyfile --adapter caddyfile"
sc.exe start caddy
```

5. Windows firewall:

```
New-NetFirewallRule -DisplayName 'HL-Spot HTTPS (Caddy)' -Direction Inbound -Pr
otocol TCP -LocalPort 80,443 -Action Allow
```

After the installation

1. Open `https://<IP address or domain>` (first certificate: a few seconds to a few minutes) and follow section 5, entering the **first-launch code**.
2. Linux and Windows: **Settings → Web interface → Listening address** = `127.0.0.1`, then restart the bot (`sudo systemctl restart hl-spot`, or the `HL-Spot` task on Windows). The bot then only accepts connections coming from Caddy.
3. Optional: if your IP address is fixed, allow only it in the Caddyfile (`remote_ip` lines, to uncomment).

Bot protections: 5 wrong passwords from the same address block it for 15 minutes; beyond 10 wrong passwords in 15 minutes, all addresses together, attempts are slowed down (wait of up to 60 seconds). Changing the password closes all other open sessions. Choose a strong password.

---

18. Support

- E-mail: HL-spot@cmails.eu

When you contact support, attach the log file (📝 **Log** page → Download the file). Never send your API wallet key, your `secret.key` file or your password.